

Plan de Corrección — Auditoría ObSave v1.0.38

Ad Astra Forge · Plugin ObSave

Fecha: 7 de septiembre de 2026

Referencia: [auditoria-obsave-v1.0.38.md](#)

Estado: Pendiente de implementación (sin release)

Objetivo

Corregir la eliminación automática de notas de productividad, aclarar el comportamiento del generador de carpetas (documentación + robustez), y restablecer la UX de atajos de teclado.

Fase P0 — Protección de notas nuevas y ledger

P0-1: Grace period para archivos recién creados

Archivo: `src/engine/SyncEngine.ts`

```
private pendingLocalPaths = new Map<string, number>();
private static readonly UPLOAD_GRACE_MS = 60_000;

markPendingUpload(vaultPath: string): void {
  this.pendingLocalPaths.set(vaultPath, Date.now());
}

private shouldProtectLocalUpload(path: string, file: TFile): boolean {
  const marked = this.pendingLocalPaths.get(path);
  if (marked && Date.now() - marked < SyncEngine.UPLOAD_GRACE_MS) {
    return true;
  }
  // Archivo creado hace menos de 60 s sin historial ledger previo
  return file.stat.ctime > Date.now() - SyncEngine.UPLOAD_GRACE_MS;
}
```

En la rama `inLocal && !inRemote && inLedger`, antes de `vault.trash`:

```
if (this.shouldProtectLocalUpload(path, localFile)) {
  const content = await this.app.vault.read(localFile);
  const driveFileId = await this.pushLocalFile(
    provider, folder.folderId, path, localFile, undefined, content,
  );
  ledger[path] = this.buildLedgerEntry(
    content, localFile.stat.mtime, driveFileId, localFile.stat.size,
  );
  this.pendingLocalPaths.delete(path);
  uploadedCount++;
  continue;
}
```

Aplicar la misma lógica en `runGitHubBidirectionalSync` con `uploadRemoteFile` .

P0-2: Marcar uploads pendientes desde `main.ts`

Archivo: `src/main.ts`

```
import { TFile } from "obsidian";

this.app.vault.on("create", (file) => {
  if (file instanceof TFile && file.extension === "md") {
    this.syncEngine.markPendingUpload(file.path);
    if (this.syncEngine.getStatus() !== "syncing") {
      void this.refreshDecoratorsImmediate();
    }
  }
});
```

P0-3: Limpiar ledger al cambiar contexto de sync

Archivos: `src/ui/ObSaveSettingTab.ts` , handlers de conexión GitHub

Al conectar GitHub, cambiar `remoteUrl` , o seleccionar carpeta Drive:

```
this.plugin.settings.syncedLedger = {};
await this.plugin.saveSettings();
```

Puntos de inserción:

Evento	Archivo	Estado v1.0.38
Picker carpeta Drive existente	<code>ObSaveSettingTab.openGoogleFolderPicker</code>	✅ Ya limpia
Desconectar proveedor	<code>main.disconnectProvider</code>	✅ Ya limpia
Conectar / configurar GitHub	<code>ObSaveSettingTab (handleSetupResult)</code>	❌ Falta
Cambiar repo GitHub (picker)	<code>ObSaveSettingTab</code>	❌ Falta

Fase P1 — Robustez del motor 3 vías

P1-1: Poda de entradas ledger huérfanas

Archivo: `src/engine/SyncEngine.ts`

Añadir rama antes del bucle principal o dentro de él:

```
if (!inLocal && !inRemote && inLedger) {
  delete ledger[path];
  continue;
}
```

P1-2: Confirmación de borrado remoto antes de trash local

Sustituir la suposición «no listado en remoto = borrado intencional» por:

1. Si `ledgerEntry.driveFileId` existe → verificar con HEAD/GET en API remota.
2. Si el recurso responde **404** → proceder con `vault.trash`.
3. Si nunca tuvo ID remoto o la verificación falla por red → **subir**, no borrar.

P1-3: ID de upload Google Drive desde respuesta HTTP

Archivo: `src/providers/GoogleDriveProvider.ts`

Tras upload multipart exitoso, parsear el JSON de respuesta (contiene `id`) en lugar de depender exclusivamente de:

```
const remoteFiles = await provider.listFiles(parentFolderId);
const created = remoteFiles.find((file) => file.name === fileName);
```

Archivo: `src/engine/SyncEngine.ts` — `pushLocalFile` debe usar el ID retornado por `uploadFile`.

Fase P2 — Atajos y badges UI

P2-1: Helper de etiqueta de atajo

Archivo nuevo: `src/utils/shortcutLabel.ts`

```
import { Platform } from "obsidian";

export function formatShortcutLabel(shortcut: string): string {
  const mod = Platform.isMacOS ? "Cmd" : "Ctrl";
  return shortcut.replace(/\bMod\b/g, mod);
}
```

Archivo: `src/ui/ObSaveSettingTab.ts`

```
import { formatShortcutLabel } from "../utils/shortcutLabel";

if (shortcut) {
  row.createEl("kbd", {
    text: formatShortcutLabel(shortcut),
    cls: "obsave-kbd",
  });
}
```

Actualizar strings pasados a `renderActionButton` para panel principal si se añade badge a Mod+Shift+O.

P2-2: Atajos menos conflictivos

Opción A (recomendada): Cambiar defaults a `Mod+Alt+*` :

```
hotkeys: [{ modifiers: ["Mod", "Alt"], key: "n" }], // Nota rápida
hotkeys: [{ modifiers: ["Mod", "Alt"], key: "m" }], // Captura
hotkeys: [{ modifiers: ["Mod", "Alt"], key: "i" }], // Informe
hotkeys: [{ modifiers: ["Mod", "Alt"], key: "o" }], // Panel
```

Opción B (conservadora): Eliminar `hotkeys` de `addCommand` y documentar en VISTA 1:

Asigna atajos en Ajustes → Atajos → busca «ObSave».

P2-3: Fallback `openObSavePanel` compatible

Archivo: `src/main.ts`

```
openObSavePanel(): void {
  this.app.setting.open();
  const setting = this.app.setting as typeof this.app.setting & {
    openTabById?: (id: string) => void;
    pluginTabs?: Array<{ id?: string; display: () => void }>;
  };
  if (typeof setting.openTabById === "function") {
    setting.openTabById(this.manifest.id);
  } else {
    setting.pluginTabs
      ?.find((tab) => tab.id === this.manifest.id)
      ?.display();
  }
  this.settingsTab.openMainPanel();
}
```

Considerar bump de `minAppVersion` si se adopta `openTabById` como requisito estricto.

Fase P3 — Generador de carpetas

P3-1: Verificación robusta de carpeta

Archivo: `src/productivity/vaultStructure.ts`

```
import type { App, TFolder } from "obsidian";

function folderExists(app: App, folderPath: string): boolean {
  const node = app.vault.getAbstractFileByPath(folderPath);
  return node instanceof TFolder;
}

export async function generateVaultTemplateFolders(app: App): Promise<string[]> {
  const created: string[] = [];
  for (const folder of VAULT_TEMPLATE_FOLDERS) {
    if (folderExists(app, folder)) continue;
    try {
      await app.vault.createFolder(folder);
    }
  }
}
```

```

        created.push(folder);
    } catch (error) {
        console.warn(`[ObSave] No se pudo crear «${folder}»:`, error);
    }
}
// ... Notices existentes
return created;
}

```

P3-2: Aviso si auto-sync está activo

Antes de generar carpetas, si hay proveedor conectado y auto-sync ON:

```

new Notice(
    "ObSave: El auto-sync está activo. Las notas sin respaldo remoto pueden verse
    afectadas por la sincronización.",
    8000,
);

```

Checklist de implementación

- ☐ P0-1 Grace period en SyncEngine (GDrive + GitHub)
- ☐ P0-2 markPendingUpload en vault.on("create")
- ☐ P0-3 Limpiar ledger al conectar/cambiar GitHub
- ☐ P1-1 Poda ledger huérfano
- ☐ P1-2 Verificación explícita de borrado remoto
- ☐ P1-3 ID upload Drive desde respuesta HTTP
- ☐ P2-1 formatShortcutLabel + badges UI
- ☐ P2-2 Atajos Mod+Alt+* o sin defaults
- ☐ P2-3 Fallback openObSavePanel
- ☐ P3-1 folderExists + try/catch
- ☐ P3-2 Aviso auto-sync en generador
- ☐ npm run build exit 0
- ☐ Verificación manual (sección 6 del informe de auditoría)
- ☐ Actualizar .cursor/decision_log.md tras implementar

Estimación de riesgo

Fase	Riesgo	Notas
P0	Medio	Cambia semántica de sync; requiere pruebas con GDrive y GitHub
P1	Medio-Alto	P1-2 toca lógica core; probar escenarios de borrado remoto intencional
P2	Bajo	Solo UX; atajos pueden requerir re-asignación manual del usuario
P3	Bajo	Defensivo; no altera sync

